

ASSIGNMENT-5

BOLISETTY SAMANVITHA

U24AI024

Consider a problem that is stated as follows. Three girls and three boys are on one side of a river, along with a boat that can hold one or two people. Find a way to get everyone to the other side without ever leaving a group of girls in one place outnumbered by the boys in that place. Formulate the problem with appropriate search space and do as per following instructions.

1. Implement and solve the problem optimally using the following search problem.

- a) Depth Limited Search (take limit=3)
- b) Iterative Deepening Search

```
total_girls=3
total_boys=3

def is_valid(g_left,b_left):
    g_right=total_girls-g_left
    b_right=total_boys-b_left

    if g_left<0 or b_left<0:
        return False

    if g_left>total_girls or b_left>total_boys:
        return False

    #left side condition
    if g_left>0 and b_left>g_left:
        return False

    #right side condition
    if g_right>0 and b_right>g_right:
        return False

    return True
```

```

def get_next_state(state):
    g_left=state[0]
    b_left=state[1]
    boat=state[2]

    next_states=[]
    moves=[(1,0),(2,0),(0,1),(0,2),(1,1)] # 1G 0B, 2G 0B, 0G 1B, 0G 2B,
1G 1B

    for move in moves:
        g_move=move[0]
        b_move=move[1]

        #if boat on left
        if boat==0:
            new_g=g_left-g_move #we subtract because we are moving from
left to right
            new_b=b_left-b_move
            new_boat=1
        else: #boat on right
            new_g=g_left+g_move #we add because we are moving from right
to left
            new_b=b_left+b_move
            new_boat=0

        if is_valid(new_g,new_b):
            next_states.append((new_g,new_b,new_boat))

    return next_states

def depth_limited_search(state,goal,limit,path,visited):
    if state==goal:
        return path

    if limit==0:
        return None

```

```

        visited.add(state)
        next_states=get_next_state(state)

        for next_s in next_states:
            if next_s not in visited:

result=depth_limited_search(next_s,goal,limit-1,path+[next_s],visited)
            if result is not None:
                return result

        visited.remove(state)
        return None

def iterative_deepening_search(initial,goal):
    depth=0
    while True:
        visited=set()
        result=depth_limited_search(initial,goal,depth,[initial],visited)
        if result is not None:
            return result

        depth+=1

initial_state=(3,3,0)
goal_state=(0,0,1)

print("Depth Limited Search with limit = 3")
result = depth_limited_search(initial_state, goal_state, 3,
[initial_state], set())

if result is None:
    print("No solution found within depth 3")
else:

```

```

    for step in result:
        print(step)

print("\nIterative Deepening Search")
solution = iterative_deepening_search(initial_state, goal_state)

print("Optimal Solution:")
for step in solution:
    print(step)

```

Output:

```

PS C:\Users\Lenovo\Desktop\Sem4\AI Lab> & C:/Python314/python.exe
"c:/Users/Lenovo/Desktop/Sem4/AI Lab/Assignment5/pro1.py"
Depth Limited Search with limit = 3
No solution found within depth 3

```

Iterative Deepening Search

Optimal Solution:

```

(3, 3, 0)
(3, 1, 1)
(3, 2, 0)
(3, 0, 1)
(3, 1, 0)
(1, 1, 1)
(2, 2, 0)
(0, 2, 1)
(0, 3, 0)
(0, 1, 1)
(1, 1, 0)
(0, 0, 1)

```

```

PS C:\Users\Lenovo\Desktop\Sem4\AI Lab>

```

2. Give comparison of both algorithms with the number of explored states and state approximate time complexities.

Depth-Limited Search (DLS):

- Explored States (limit=3): 10
- Time Complexity: $O(b^d)$, where b is branching factor and d is depth limit.

- DLS is not complete and not optimal if the solution is deeper than the limit.

Iterative Deepening Search (IDS):

- Explored States: 163
- Time Complexity: $O(b^d)$, where b is branching factor and d is the depth of the solution.
- IDS is complete and optimal like Breadth-First Search, but with the memory benefits of Depth-First Search.

PS C:\Users\Lenovo\Desktop\Sem4\AI Lab>